

本章知识要点:

- 栈和队列的基本概念。
- 栈和队列的顺序存储结构和链式存储结构。
- 栈和队列的应用。

3.1 栈

栈是一种特殊的线性表，它的逻辑结构和存储结构与线性表相同，其特殊性体现在“运算受限”，即无论往表中插入元素还是删除表中已有元素，都被限制在线性表的一端进行。一般将表尾作为操作端，如图 3.1 所示。

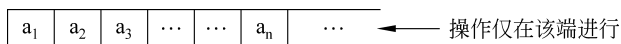


图 3.1 操作仅在表一端进行

3.1.1 栈的定义

栈是限制在表的一端进行插入和删除操作的线性表。能够进行操作的一端是浮动端，称为**栈顶**，通常用一个“栈顶指针”指示，它的位置会随着操作而发生变化；与此相对，表的另一端是固定端，称为**栈底**。如同线性表可以为空表一样，当栈中没有元素时称为空栈。往栈中插入元素的操作称为**入栈**，删除栈中元素的操作称为**出栈**。图 3.2 表示了一个栈。

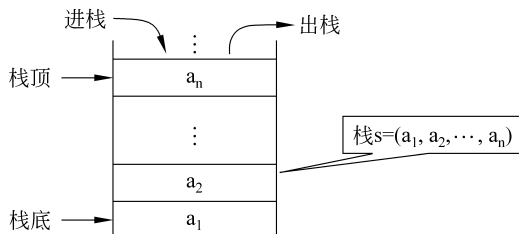


图 3.2 栈的示意图

根据栈的运算特性，所有操作都只在栈顶进行。如果将数据元素按照 $a_1, a_2, a_3, \dots, a_n$ 的顺序依次入栈，则此时 a_1 在栈底， a_n 在栈顶，如图 3.2 所示。当要取出数据时，则必须按 $a_n, a_{n-1}, \dots, a_1$ 的顺序进行。由此可知， a_n 作为栈顶元素总是最后入栈的，而最先出栈； a_1 作为栈底元素总是最先入栈的，而最后出栈。栈按照这种后进先出 (Last In First Out,

LIFO) 或者先进后出 (First In Last Out, FILO) 的原则来组织数据的, 因此, 它也被称为后进先出或先进后出的线性表。

基于栈的特性, 栈的抽象数据类型定义如下:

```
ADT Stack {  
    数据对象 D:  $D = \{ a_i | a_i \in \text{ElemSet}, i = 1, 2, \dots, n, n \geq 0 \}$   
    数据关系 R:  $R = \{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i = 2, \dots, n \}$   
    约定  $a_n$  端为栈顶,  $a_1$  端为栈底。  
    基本操作 P:  
    InitStack(&S)  
        操作结果: 构造一个空栈 S  
    StackEmpty(S)  
        初始条件: 栈 S 已存在  
        操作结果: 若栈 S 为空栈, 则返回 TRUE, 否则返回 FALSE  
    StackFull(S)  
        初始条件: 栈 S 已存在  
        操作结果: 若栈 S 满, 则返回 TRUE, 否则返回 FALSE  
    GetTop(S, &e)  
        初始条件: 栈 S 已存在  
        操作结果: 若栈 S 非空, 用 e 返回 S 的栈顶元素  
    Push(&S, e)  
        初始条件: 栈 S 已存在  
        操作结果: 若栈 S 不满, 插入元素 e 为新的栈顶元素  
    Pop(&S, &e)  
        初始条件: 栈 S 已存在且非空  
        操作结果: 若栈 S 非空, 删除 S 的栈顶元素, 并用 e 返回其值  
} ADT Stack
```

3.1.2 栈的顺序存储结构

采用顺序存储结构的栈称为顺序栈, 它需要一段连续的存储空间来存储栈中元素。

1. 顺序栈的类型定义

类似于顺序表的定义, 通常用预先设定的足够大的一维数组来存放数据。由于栈顶会随着插入和删除操作而发生变化, 用整型变量 **top** 作为栈顶指针指示栈顶的当前位置。栈底位置固定不变, 可以设置在数组的任意端, 一般将数组的下标端作为栈底。

顺序栈的类型描述如下:

```
#define MAXSIZE 100  
typedef struct  
{ Elemtype data[MAXSIZE];  
    int top;  
} SqStack;
```

在顺序栈中, 用于指示栈顶的当前位置的 **top** 是整型, 它的实质是栈顶元素在数组

中的下标。栈顶指针 top 直接反映出栈的当前状态：空栈时，栈顶指针 top 为 -1 ；栈满时，栈顶指针 top 为 $\text{MAXSIZE}-1$ ；入栈时，栈顶指针 top 加 1；出栈时，栈顶指针 top 减 1。

如图 3.3 所示为 MAXSIZE 为 6 的顺序栈中数据元素和栈顶指针 top 的变化情况。其中，图 3.3(a)表示空栈；图 3.3(b)表示元素 a 进栈， top 指示的是当前栈顶的位置；图 3.3(c)表示 b 、 c 、 d 、 e 、 f 依次进栈后栈满的情况；元素 f 、 e 、 d 依次出栈后的情况如图 3.3(d)所示， top 为当前栈顶元素 c 的位置；图 3.3(e)表示元素 c 、 b 、 a 继续出栈又重新回到空栈状态。

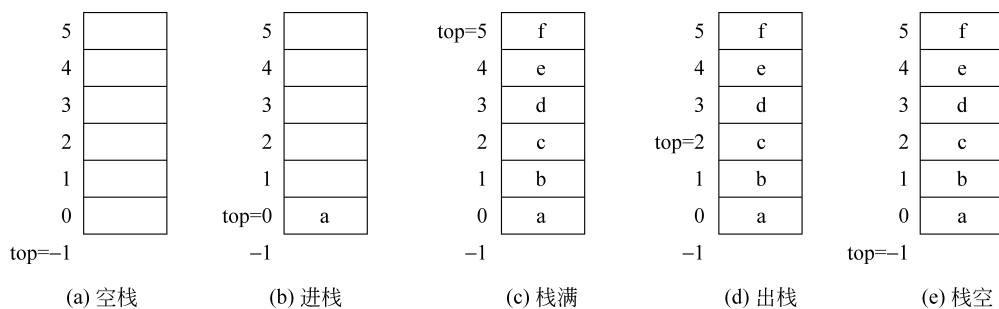


图 3.3 栈顶指针 top 与栈中数据元素的关系

2. 顺序栈的算法实现

(1) 初始化栈（置栈空）。

初始化栈主要是分配存储空间，并将栈顶指针置为 -1 。

```
int InitStack(SqStack &S)
//构造一个空栈
{
    S.top=-1;
    return OK;
}
```

(2) 判栈空。

```
int StackEmpty(SqStack S)
//判栈为空栈时返回值为真,反之则为假
{    return(S.top==-1? TRUE:FALSE); }
```

(3) 判栈满。

```
int StackFull(SqStack S)
//判栈为满栈时返回值为真,反之则为假
{    return(S.top==MAXSIZE-1?TRUE:FALSE); }
```

(4) 进栈。

进栈时应首先判栈满，若栈不满则将栈顶指针 top 上移，存入元素。

```
int Push(SqStack &S, Elemtype e)
{
    //将元素 e 插入到栈中,作为新栈顶
```

```

    if(StackFull(S)) return ERROR;           //栈满
    S.top++;                                 //top 加 1, 栈顶位置上移
    S.data[S.top]=e;                         //数据 e 存入当前栈顶
    return OK;
}

```

(5) 出栈。

出栈时应首先判断栈是否为空, 若栈不为空, 则取出栈顶元素, 将栈顶指针 `top` 下移, 再返回栈顶元素。

```

int Pop(SqStack &S, Elemtyp e)
{
    if(StackEmpty(S)) return ERROR;         //若栈不为空, 则删除栈顶元素
    e=S.data[S.top];                         //栈空
    S.top--;                                 //取出数据放入 e 所指单元中
    return OK;                               // top 减 1, 栈顶位置下移
}

```

(6) 取栈顶元素。

```

int GetTop(SqStack S, Elemtyp e)
{
    if(StackEmpty(S)) return ERROR;         //若栈不为空, 则取栈顶元素
    e=S.data[S.top];                         //栈空
    return OK;                               //取出数据, top 不变
}

```

由于栈的运算特殊性, 顺序栈中进栈和出栈操作并不存在移动数据的问题, 因而效率较高。但顺序栈需要预先估计准确的存储空间大小, 需要预先分配一个较大空间, 这有可能造成存储空间的浪费。

3.1.3 栈的链式存储结构

若是栈中元素的数目变化范围较大或不清楚栈元素的数目, 就应该考虑使用链式存储结构。用链式存储结构表示的栈称为“链栈”。

1. 链栈的类型定义

链栈通常用一个无头结点的单链表表示, 其结点结构与单链表的结点结构相同。

```

typedef struct node
{
    Elemtyp data;
    struct node* next;
}StackNode, *LinkStack;
LinkStack top;           //top 为栈顶指针

```

由于栈中的主要运算是在栈顶进行插入、删除, 对于单链表来说, 在表头插入和删除结点要比在表尾相对简单, 因此将单链表表头作为栈顶, 则单链表的头指针即为栈顶

指针。通常将链栈表示如图 3.4 所示。

2. 链栈的算法实现

链栈的本质是简化的单链表, **top** 作为栈顶指针始终指向链表首结点。进栈操作就是在链表表头插入一个新的结点, 出栈操作就是删除当前的表头结点并释放空间。

(1) 初始化栈(置空栈)。

```
LinkStack InitStack()
//空栈的 top 指针为 NULL
{ return NULL; }
```

(2) 判栈空。

```
int StackEmpty(LinkStack top)
//判栈为空栈时返回值为真, 反之为假
{ return (top==NULL? TRUE:FALSE); }
```

(3) 进栈。

```
void Push(LinkStack top, Elemtyp e)
{ //将元素 e 进链栈, 即在表头插入新的结点
  StackNode *s;
  s=(StackNode*)malloc(sizeof(StackNode));
  s->data=e;
  s->next=top;
  top=s;
}
```

(4) 出栈。

```
int Pop(LinkStack top, Elemtyp &e)
{ //若栈不为空将栈顶元素出栈, 即为删除表头结点
  StackNode *p;
  if(StackEmpty(top)) return ERROR; //栈空
  e=top->data;
  p=top;
  top=top->next;
  free(p);
  return OK;
}
```

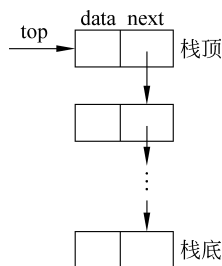


图 3.4 链栈示意图

3.2 栈的应用举例

根据栈的运算特点, 在很多实际问题中都利用栈作为一个辅助的数据结构来进行求解, 下面通过几个例子进行说明。

3.2.1 数制转换

数制转换问题是指对于输入的任意一个非负十进制整数 N ，打印输出与其等值的 r 进制数。一般利用辗转相除法解决这个问题，以 $N=1348$ ， $r=8$ 为例，其转换方法如下：

N	$N/8$ (整除)	$N\%8$ (求余)	
1348	168	4	低
168	21	0	
21	2	5	
2	0	2	高

结果为 $(1348)_{10}=(2504)_8$ 。

对其计算过程进行分析可以发现，八进制的各位数实际上就是每次运算所得的余数值，它们产生的顺序是由低位到高位，这恰好与输出顺序相反。因此，在转换过程中每得到一位余数就将其进栈保存，转换完毕后再依次出栈，即可得出结果。

将十进制数 N 转换为 r 进制数，算法思路如下：

(1) 当 $N \neq 0$ ，则重复执行①和②：

① 将 $N\%r$ 进栈 s ；

② $N = N/r$ 。

(2) 将栈 s 的内容依次出栈，算法结束。

具体实现如算法 3.1 所示。

```
typedef int Elemtyp;
void conversion(int N, int r)
{ SqStack S;                      //定义一个顺序栈 s
  Elemtyp x;
  InitStack(S);
  while(N)
  { Push(S, N%r);                 //余数入栈
    N=N/r;
  }
  while(!StackEmpty(S))
  { Pop(S, x);
    printf("%d", x);
  }
}
```

算法 3.1

3.2.2 括号匹配的检验

假设在一个算术表达式中，可以包含三种括号：圆括号“(”和“)”，方括号“[”和“]”和花括号“{”和“}”，并且这三种括号可以按任意的次序嵌套使用，例如某一个算术表达式中的括号使用括号情况为“ $\cdots[\cdots\{\cdots\}\cdots[\cdots]\cdots[\cdots]\cdots(\cdots)\cdots$ ”。括号匹

配的检验主要就是判别给定表达式中所含括号是否正确配对。

算术表达式中各种括号的使用规则为：出现左括号，必有相应的右括号与之匹配，并且每对括号之间可以嵌套，但不能出现交叉情况。根据该规则可知，当前最晚出现的左括号总是最先与随后出现的右括号进行匹配，这一规律与栈的“后进先出”运算特性吻合，因此可以利用栈来解决这一问题，算法思路如下：

依次输入表达式每一个字符，若是左括号，将其入栈保存；若是右括号，则出栈左括号，并检验与其是否匹配。循环执行，直到表达式输入结束。

在检验过程中，若遇到以下几种情况之一，就可以得出括号不匹配的结论。

- 当遇到某一个右括号时，栈已空，说明到目前为止，右括号多于左括号；
- 从栈中弹出的左括号与当前检验的右括号类型不同，说明出现了括号交叉情况；
- 算术表达式输入完毕，但栈中还有没有匹配的左括号，说明左括号多于右括号。

具体实现如算法 3.2 所示。

```
typedef char Elemtyp;
int bracketmatching()
{
    SqStack S;
    Elemtyp ch;
    InitStack(S);
    while ((ch=getchar())!='\n')
    {
        switch (ch)    //遇左括号入栈;遇到右括号时,分别检测匹配情况
        {
            case '(':
            case '[':
            case '{':
                Push(S,ch);break;
            case ')':
                if (StackEmpty(S)) return FALSE;
                else
                { Pop(S,ch); if (ch!= '(') return FALSE; }
                break;
            case ']':
                if (StackEmpty(S)) return FALSE;
                else
                { Pop(S,ch); if (ch!= '[') return FALSE; }
                break;
            case '}':
                if (StackEmpty(S)) return FALSE;
                else
```

算法 3.2

```
        { Pop(S,ch); if (ch!='{') return FALSE; }  
        break;  
        default:break;  
    }  
}  
if (StackEmpty(S)) return TRUE;  
else return FALSE;  
}
```

算法 3.2 (续)

3.2.3 简单表达式求值

表达式是由运算对象、运算符、括号组成的有意义的式子，而表达式求值是程序设计语言编译器中一个最基本的问题，它是栈的一个典型的应用实例。为了讨论方便，在不失一般性的前提下，以 10 以内整数构成的四则运算表达式为例来说明栈在表达式求值中的作用。

1. 表达式的定义

(1) 表达式的构成。

在计算机中，任何一个表达式都是由操作数 (operand)、运算符 (operator) 和界限符 (delimiter) 组成的。本节讨论的简单表达式中，其操作数由数字 0~9 构成，运算符包括加、减、乘、除和括号，表达式界限符设定为“#”。由此，将表达式的构成分成两类符号：操作数和算符，其中算符包含运算符和界限符。

(2) 表达式的运算规则。

- 先乘除、后加减；
- 同级运算时先左后右；
- 先括号内，后括号外。

2. 算符的优先级

根据运算规则，在运算的每一步中，任意两个相继出现的算符 θ_1 和 θ_2 之间的优先关系主要有以下三种情况。

- $\theta_1 < \theta_2$: θ_1 的优先级低于 θ_2 。
- $\theta_1 = \theta_2$: θ_1 的优先级等于 θ_2 。
- $\theta_1 > \theta_2$: θ_1 的优先级高于 θ_2 。

如表 3-1 所示，其中，当 θ_1 和 θ_2 都为“+”时，例如 $2+3+4$ ，应该先算由 θ_1 代表的先出现的第一个加法，再算随后出现的由 θ_2 代表的第二个加法，也就是说，相同的算符左优先。出现“E”表示错误，即 θ_1 和 θ_2 不可能相继出现，例如“ $\cdots 2)7(\cdots$ ”。为方便处理，在表达式的前后分别加上一个“#”作为开始和结束符，它的优先级最低。

3. 算法思路

(1) 算符优先关系的描述与实现。

根据表 3-1，对于相继出现的算符 θ_1 和 θ_2 它们之间的优先关系可以总结为以下 5 种

情况。

表 3-1 算符优先关系表

$\theta_1 \backslash \theta_2$	+	-	*	/	(	)	#
+	>	>	<	<	<	>	>
-	>	>	<	<	<	>	>
*	>	>	>	>	<	>	>
/	>	>	>	>	<	>	>
(	<	<	<	<	<	=	E
)	>	>	>	>	E	>	>
#	<	<	<	<	<	E	=

- θ_1 是 “+” 和 “-” 时，当 θ_2 是 “*”、 “/” 和 “(” 时，它们的优先级关系应该是 “<”，其余时候它们之间的优先关系是 “>”；
- θ_1 是 “*” 和 “/” 时，当 θ_2 是 “(” 时，它们之间的优先级是 “<”，其余时候是 “>”；
- θ_1 是 “(” 时，当 θ_2 是表达式结束符时，它们之间的优先级是错误的可以用字符 “E” 表示，其余情况又可以分成两种：当 θ_2 是 “)” 时，它们之间的优先级是相同的 “=”， θ_2 是其他字符时，它们之间的优先关系是 “<”；
- θ_1 是 “)” 时，当 θ_2 是 “(” 时，它们之间的优先关系是错误的用字符 “E” 表示， θ_2 是其他字符时，它们之间的优先关系就是 “>”；
- θ_1 是表达式结束符时，当 θ_2 也是表达式结束符时，它们之间的优先关系时相等的用 “=” 表示，其余情况又可分为两种：当 θ_2 是 “)” 时，它们之间的优先级关系用 “E” 表示， θ_2 是其他字符时，它们之间的优先级关系都可以利用 “<” 来表示。

这 5 种情况是根据 θ_1 的值来分的，即当 θ_1 的值确定后，再根据 θ_2 的值来判断。由此，利用多分支的选择结构可以实现算符之间的优先关系。

(2) 算法步骤。

表达式作为满足一定语法规则的字符串，求值时需要自左向右地进行扫描。例如表达式 “4+2*(3+6/2)”，当扫描到 4+2 时是不能立刻计算的，因为后面还有优先级更高的运算。此时，需要将操作数和算符暂时保存，根据后面出现的算符的优先级高低进行计算。

表达式求值的处理过程主要包含以下 3 步：

- ① 设置两个工作栈：optr 算符栈和 opnd 操作数栈。初始置 opnd 为空栈；起始符 “#” 为 optr 的栈底元素；
- ② 自左向右扫描表达式中的每个字符 c：
 - 若 c 为操作数，则进 opnd 栈；
 - 若 c 为算符，则让 optr 栈的栈顶元素与 c 比较优先级：

- 若栈顶算符优先级低于刚读入的运算符 c ，则让刚读入的运算符 c 进 $optr$ 栈。
- 若栈顶算符优先级高于刚读入的运算符 c ，则将栈顶算符退栈，送入 θ ；同时将操作数栈 $opnd$ 退栈两次，得到两个操作数 a 、 b ，对 a 、 b 进行 θ 运算后，将运算结果作为中间结果推入 $opnd$ 栈。
- 若栈顶运算符的优先级与刚读入的运算符 c 相同，说明左右括号相遇，只需将栈顶运算符（左括号）退栈即可。

③ 直到扫描到 c 为定界符，即 $optr$ 栈的栈顶元素和当前读入的字符均为 “#”，则整个表达式求值完毕。

具体实现如算法 3.3 所示。

```
float CaculateExpression()
{ //算术表达式求值的算符优先算法。设 optr 和 opnd 分别为算符栈和操作数栈，
  //OP 为算符集合
  InitStack(optr); Push(optr, '#');
  InitStack(opnd); c=getchar();
  while(c!='#' || GetTop(optr)!='#')
  {
    if(!In(c, OP)) {Push(opnd, c); c=getchar();} //不是算符则进栈
    else
      switch(Precede(GetTop(optr), c))
      {
        case '<': //栈顶算符优先级低
          Push(optr, c); c=getchar();
          break;
        case '=': //优先级相同，脱去括号并读入下一字符
          Pop(optr, x); c=getchar();
          break;
        case '>': //栈顶算符优先级高，退栈并将运算结果入栈
          Pop(optr, theta);
          Pop(opnd, b); Pop(opnd, a);
          Push(opnd, Operate(a, theta, b));
          break;
      }
  }
  return GetTop(opnd);
}
```

算法 3.3

算法中还调用了两个函数。其中 **Precede** 函数是判定算符栈栈顶算符与读入的算符之间优先级的函数；**Operate** 函数为进行二元运算的函数。

(3) 实例演示。

例如，对表达式 “4+2*(7-3)” 进行求值，求值过程中两个工作栈的使用情况如图 3.5

所示。图 3.5(a)呈现出两个工作栈的初始状态。从左到右扫描表达式,依次读入“4”、“+”、“2”、“*”、“(”、“7”、“-”、“3”。很容易判断出“4”、“2”、“7”、“3”是操作数依次进 opnd 栈;当读入的是算符时,需要将它与栈顶运算符比较优先级。在依次读入“+”、“*”、“(”、“-”算符时,其优先级都比栈顶运算符的优先级高,因此将其依次进 optr 栈,如图 3.5(b)所示。紧接着读入的字符是“)”,它的优先级比当前的栈顶算符“-”优先级低,因此需要进行出栈运算。将 opnd 栈中的操作数“3”和“7”依次出栈,将 optr 栈中的“-”出栈,计算出 $7-3$ 的结果进 opnd 栈以便继续参与后续计算,如图 3.5(c)和(d)所示。继续将当前读入的字符“(”与栈顶算符“(”比较优先级,由于它们的优先级相同,则需要脱掉该层括号,即将“(”出栈,并继续向右扫描表达式读入下一个字符“#”,如图 3.5(e)所示。已知“#”的优先级比“*”和“+”的优先级低,因此进行了两次的出栈运算,并把运算结果都保存在 opnd 栈中,如图 3.5(f)和(g)所示。值得注意的是,在图 3.5(g)中 optr 栈中的栈顶元素和当前读入的字符均为表达式的定界符“#”,此时意味着整个表达是求值完毕,而最终结果就保存在 opnd 栈的栈顶元素中。

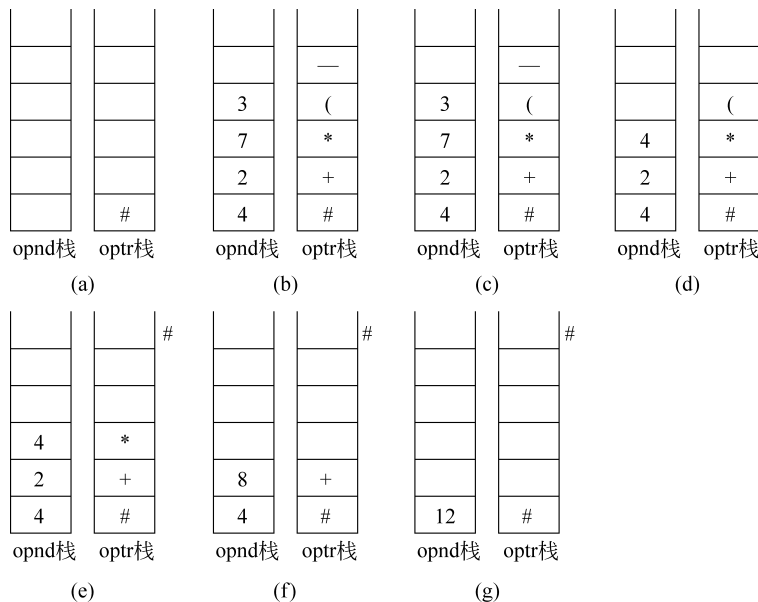


图 3.5 表达式求值过程中各工作栈中的情况

表 3-2 中列出了表达式“ $4+2*(7-3)$ ”求值过程中的每一个步骤和操作,清晰地体现了栈的相关运算在这个过程中的应用。

表 3-2 表达式的求值过程

步骤	optr 栈	opnd 栈	依次读入字符	主要操作
1	#		$4+2*(7-3)\#$	Push(opnd, '4')
2	#	4	$+2*(7-3)\#$	Push(optr, '+')
3	# +	4	$2*(7-3)\#$	Push(opnd, '2')
4	# +	4 2	$*(7-3)\#$	Push(optr, '*')
5	# + *	4 2	$(7-3)\#$	Push(optr, '(')

续表

步骤	optr 栈	opnd 栈	依次读入字符	主要操作
6	# + * (	4 2	7-3)#	Push(opnd,'7')
7	# + * (	4 2 7	-3)#	Push(optr,'-')
8	# + * (-	4 2 7	3)#	Push(opnd,'3')
9	# + * (-	4 2 7 3	)#	Pop(optr);Pop(opnd); Pop(opnd); Push(opnd,operate('7','-','3'))
10	# + * (-	4 2 4	)#	Pop(optr);
11	# + *	4 2 4	#	Pop(optr);Pop(opnd); Pop(opnd); Push(opnd,operate('2','*','4'))
12	# +	4 8	#	Pop(optr);Pop(opnd); Pop(opnd); Push(opnd,operate('4','+','8'))
13	#	12	#	Gettop(opnd);

表达式中运算符放在两个操作数中间的，被称为中缀表达式。通过上述求值过程可以看出，中缀表达式的计算需要考虑运算符的优先关系，计算过程比较复杂。为了处理方便，通常把中缀表达式首先转换成等价的后缀表达式。在后缀表达式中，运算符总是在操作数之后，并且没有括号，所有的计算按运算符出现的顺序，严格从左向右进行，而不用考虑运算规则和优先级。中缀表达式“4+2*(7-3)”的后缀表达式是“4273-*+”。

由于后缀表达式中既无括号又无优先级的约束，其求值算法要简单很多，只需要借助一个操作数栈即可完成求值过程。具体做法是，当从左向右扫描表达式时，每遇到一个操作数就将其进栈保存，每遇到一个运算符就从栈中取出两个操作数进行当前的计算，并将计算结果继续入栈，直到整个表达式结束，这时栈顶元素的值就是表达式的结果。

3.3 队 列

与栈类似，队列（Queue）也是一种运算受限的线性表，它的特殊性主要体现在插入和删除操作分别被限定在表的两端进行。图 3.6 就反映了这种运算受限的性质。

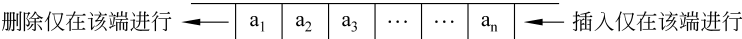


图 3.6 插入和删除分别限制在表的两端进行

3.3.1 队列的定义

与栈的“后进先出”不同，队列具有“先进先出”（First In First Out, FIFO）的特性：数据元素的插入只能在表的一端进行，而删除数据元素只能在表的另一端进行。允许删除的一端称为队头，允许插入的一端称为队尾，它们的位置会随着插入和删除操作而发生变化，分别设置“队头指针”和“队尾指针”来指示队头和队尾的位置。同线性表可

以为空表一样,当队列中没有元素时称为空队。在队尾插入元素称为入队,在队头删除元素称为出队。如图 3.7 所示是一个具有 5 个元素的队列。进队的顺序依次为 a_1, a_2, a_3, a_4, a_5 , 出队的顺序依然是 a_1, a_2, a_3, a_4, a_5 。当前的队头元素为 a_1 时,队尾元素为 a_5 。

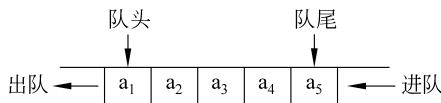


图 3.7 队列示意图

基于队列的特性,队列的抽象数据类型定义如下:

```

ADT Queue{
    数据对象:  $D=\{a_i \mid a_i \in \text{ElemSet}, i=1, 2, \dots, n, n \geq 0\}$ 
    数据关系:  $R1=\{\langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2, \dots, n\}$ 
        约定其中  $a_1$  端为队头,  $a_n$  端为队尾
    基本操作:
        InitQueue (&Q)
            操作结果: 构造一个空队列 Q
        QueueEmpty (Q)
            初始条件: 队列 Q 已存在
            操作结果: 若 Q 为空队列, 则返回 TRUE, 否则 FALSE
        QueueLength (Q)
            初始条件: 队列 Q 已存在
            操作结果: 返回 Q 的元素个数, 即队列的长度
        GetHead (Q, &e)
            初始条件: Q 为非空队列
            操作结果: 用 e 返回的队头元素
        EnQueue (&Q, e)
            初始条件: 队列 Q 已存在
            操作结果: 插入元素 e 为 Q 的新的队尾元素
        DeQueue (&Q, &e)
            初始条件: Q 为非空队列
            操作结果: 删除 Q 的队头元素, 并用 e 返回其值
}ADT Queue
    
```

3.3.2 队列的顺序存储结构

与顺序栈类似,用顺序存储的队列就称为顺序队列。

1. 顺序队列的类型定义

顺序队列需要预先分配一段连续空间来存储队列中数据元素队列的顺序存储结构,一般情况下借助于一维数组来作为队列的顺序存储空间。随着元素的出队、入队,队头和队尾的位置不断发生变化,设置头指针和尾指针可以反映出当前队列队头和队尾的位置信息。通常约定队头指针 **front** 指示队头元素前面一个位置,队尾指针 **rear** 指示队尾元素的位置。由于顺序队列使用一维数组存放数据,队列中各个元素的位置信息可以用下

标来表示，因此与顺序栈的栈顶指针 **top** 类似，在顺序队列中 **front** 和 **rear** 也设置为整型变量。

顺序队列类型描述如下：

```
#define MAXSIZE 100
typedef struct
{ Elemtype data[MAXSIZE];
  int front,rear;
}SqQueue;
```

假设有 **SqQueue *p**，则 **p** 为指向顺序队列的指针。**p->front** 和 **p->rear** 分别为队头指针和队尾指针。根据约定，**p->data[p->front+1]**表示队头元素，**p->data[p->rear]**表示队尾元素。

队列初始化时，令 **p->front=p->rear=-1**。当新元素进队时，队尾指针 **p->rear** 增 1；当有元素出队时，队头指针 **p->front** 增 1。图 3.8(a)表示具有 8 个存储空间的空队列状态，随后三个元素依次进队，如图 3.8(b)所示；经过若干次进队、出队后顺序队列的状态如图 3.8(c)所示。最后，元素 **h** 入队，状态如图 3.8(d)所示。

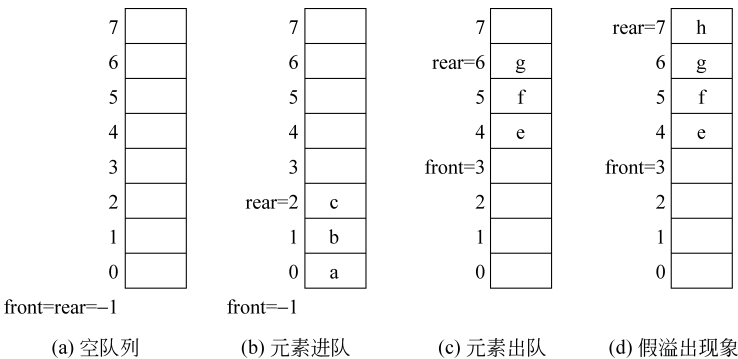


图 3.8 队尾指针 **front** 和队尾指针 **rear** 与顺序队列中数据的关系

2. 循环队列

从图中可以看出，随着进队、出队的进行，整个队列会向后移动，最终会到达空间的最后端，即 **rear=MAXSIZE-1**，此时即使前端还有空闲位置，再有入队操作也会发生溢出。但事实上，此时的队列中并未真的“满员”，还有空闲存储空间，这种现象称为“假溢出”，如图 3.7(d)所示。

假溢出现象的发生是由于“队尾进队队头出队”这种受限制的操作造成的，不论是进队还是入队，指针的操作只增不减，最终总会到达存储空间的后端。解决假溢出的方法主要有两种：

- 采用平移元素的方法。当发生假溢出时，就把整个队列的元素平移到存储区的首部，然后再插入新元素。这种方法的优点是简单，缺点是可能造成大量元素的移动，效率很低。
- 将顺序队列的存储区假想为一个首位相接的环状的空间，当发生假溢出时，将新

元素插入到第一个位置上。这样做,虽然物理上队尾在队首之前,但逻辑上队首仍然在前,头尾指针的关系不变。进队和出队仍按“先进先出”的原则进行,这就是循环队列,如图 3.9 所示。很显然,该方法不需要移动元素,操作效率高,空间的利用率也很高。

假设为队列分配的数组单元数为 MAXSIZE ,在 C 语言中,它的下标在范围 $0 \sim \text{MAXSIZE}-1$ 。若在增加队头或队尾指针时使其能够“首尾相接”,可以利用取模运算来实现。如下所示:

```
front=(front+1)%MAXSIZE;
rear=(rear+1)%MAXSIZE;
```

当 front 或 rear 为 $\text{MAXSIZE}-1$ 时,上述两个公式计算的结果就为 0。这样,就使得指针自动由最后面转到最前面,形成了循环的效果。

(1) 循环队列中队满与队空的判定

图 3.10 中是具有 8 个存储空间的循环队列。图 3.10(a)所示,队列中有 4 个元素,随着 $a_7 \sim a_{10}$ 的相继入队,队列中共有 9 个元素,已占满所有的存储空间,此时 $\text{front}=\text{rear}=2$,如图 3.10(b)所示,可见在队满的情况下: $\text{front}=\text{rear}$ 。紧接着让 $a_3 \sim a_{10}$ 相继出队,此时队列变空 $\text{front}=\text{rear}=2$,如图 3.10(c)所示,可见在队空的情况下也有: $\text{front}=\text{rear}$ 。也就是说“队满”和“队空”的条件是相同的,仅凭 $\text{front}=\text{rear}$ 无法区分当前队列是满还是空。

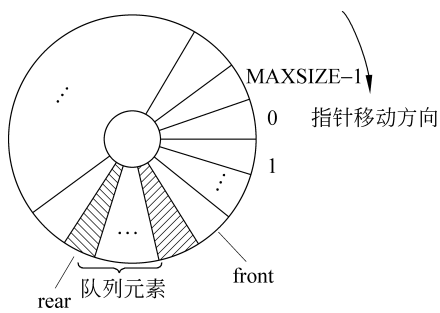


图 3.9 循环队列示意图

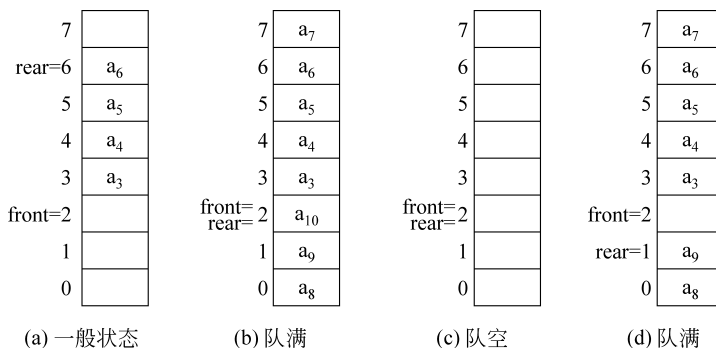


图 3.10 循环队列操作示意图

循环队列中常用以下两种方法来判定队满与队空:

- 设置一个量 len , 专门用于记录队列元素个数。当 len 为 0 时队空, 当 len 为 MAXSIZE 时队满。
- 队空条件为 $\text{rear}=\text{front}$; 当 $(\text{rear}+1)\% \text{MAXSIZE}=\text{front}$ 时队满, 即牺牲一个存储空间, 当 rear 只差一步就追上 front 时则认为队满, 如图 3.10(d)所示。

(2) 循环队列求队列长度的问题

由于循环队列“首尾相接”的结构特性, 求队列长度时会遇到两种情况。在图 3.10(a)

中, 由于 $\text{rear} > \text{front}$, 所以直接使用 $\text{rear} - \text{front}$ 可以得出该队列中当前具有 4 个元素。但当出现如图 3.10(d)所示的情况时, 由于 $\text{rear} < \text{front}$, 使用 $\text{rear} - \text{front}$ 不能求出队列长度。对于该问题, 可以有两种解决方案:

- 设置一个量 len , 专门用于记录队列元素个数;
- 当 $\text{rear} \geq \text{front}$ 时, 队列长度为 $\text{rear} - \text{front}$; 当 $\text{rear} < \text{front}$ 时, 队列长度为 $\text{rear} - \text{front} + \text{MAXSIZE}$; 因此, 循环队列的队列长度可以统一写为 $(\text{rear} - \text{front} + \text{MAXSIZE}) \% \text{MAXSIZE}$ 。

3. 循环队列的算法实现

(1) 初始化。

循环队列的初始化主要是分配存储空间, 并将队头队尾指针置为相同。

```
int InitQueue(SqQueue &Q)
{    //构造一个空队列
    Q.front=Q.rear=0;
    return OK;
}
```

(2) 判队空。

```
int QueueEmpty(SqQueue Q)
//队空时返回值为真, 反之为假
{    return(Q.front==Q.rear? TRUE:FALSE); }
```

(3) 判队满。

```
int QueueFull(SqQueue Q)
//队满时返回值为真, 反之为假
{    return((Q.rear+1)%MAXSIZE==Q.front?TRUE:FALSE); }
```

(4) 进队。

```
int EnQueue(SqQueue &Q, Elemtype e)
{
    if(QueueFull(Q)) return ERROR;           //队满
    Q.rear=(Q.rear+1)%MAXSIZE;                //rear 加 1, 队尾位置上移
    Q.data[Q.rear]=e;                         //数据 e 存入当前队尾
    return OK;
}
```

(5) 出队。

```
int DeQueue(SqQueue &Q, Elemtype &e)
{
    if(QueueEmpty(Q)) return ERROR;           //队空
    Q.front=(Q.front+1)%MAXSIZE;              //front 加 1, 队头位置上移
}
```



```

        e=Q.data[Q.front];                //取出数据放入 e 所指的单元中
        return OK;
    }

```

(6) 取队头。

```

int QueueFront(SqQueue Q,Elemtype &e)
{
    if(QueueEmpty(Q)) return ERROR;      //队空
    e=Q.data[(Q.front+1)%MAXSIZE];       //取出队头数据
    return OK;
}

```

(7) 求队列长度。

```

int QueueLength(SqQueue Q)
{
    return (Q.rear-Q.front+MAXSIZE)%MAXSIZE;
}

```

3.3.3 队列的链式存储结构

链式存储的队列称为链队列。和链栈类似，通常用单链表来实现链队列。

1. 链队列的类型定义

根据队列“先进先出”的特性，为了能够方便地在队头删除元素，在队尾插入元素，除了头指针外，还需要一个尾指针，用来指向链表的尾结点。因此，一个链队列就需要两个指针才能唯一确定，它们分别指示队头和队尾（分别称为头指针和尾指针）。

在 C 语言中，队列链式存储结构的类型定义为：

```

typedef struct node
{
    Elemtype data;
    struct node* next;
}QueueNode;                //链队列的结点类型
typedef struct
{
    QueueNode *front;       //链队列的队头指针
    QueueNode *rear;        //链队列的队尾指针
}LinkQueue;                //链队列

```

此外，为了简化链表边界条件的处理，在链队列的队头结点之前附加一个头结点，并令队头指针 front 指向附加的头结点。若定义一个指向链队列的指针：LinkQueue *q，则带头结点的链队列如图 3.11 所示。

2. 链队列的算法实现

从图 3.11 中可以看出，空的链队列的判别条件为头指针和尾指针均指向附加头结点。链队列的入队操作即为在链表表尾插入新结点；出队操作即为在链表表头删除表头结点；求链队列的队列长度就是计算链队列结点数。

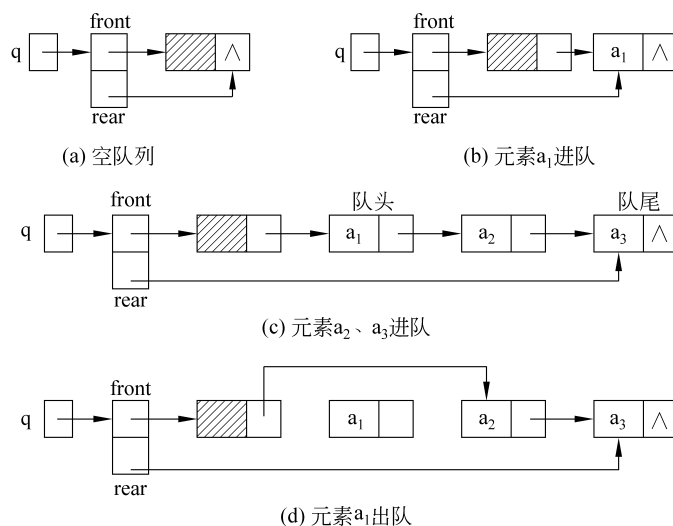


图 3.11 链队列示意图

(1) 初始化。

```

LinkQueue* InitQueue()
//创建一个带头结点的空队
{
    LinkQueue *q;
    QueueNode *p;
    q=(LinkQueue*)malloc(sizeof(LinkQueue));           //申请头尾指针结点
    p=(QueueNode*)malloc(sizeof(QueueNode));           //申请链队列头结点
    p->next=NULL;
    q->front=q->rear=p;
    return q;
}

```

(2) 判队空。

```

int QueueEmpty(LinkQueue *q)
//链队列为空时返回值为真,反之则为假
{
    return(q->front==q->rear?TRUE:FALSE);
}

```

(3) 进队。

```

void EnQueue(LinkQueue *q, Elemtype e)
{
    //将元素e进队,即在表尾插入新的结点
    QueueNode *s;
    s=(QueueNode*)malloc(sizeof(QueueNode));
    s->data=e;
    s->next=NULL;
    q->rear->next=s;
    q->rear=s;
}

```

```
}
```

(4) 出队。

在进行出队操作时,除了考虑空队情况外,还应当注意队列中只有一个元素时的情况,如图 3.11(b)所示。唯一的元素一旦出队,则队列为空,此时就需要修改队尾指针。

```
int DeQueue(LinkQueue *q,Elemtype &e)
{    //若队列不为空将出队,即为删除队头结点
    QueueNode *p;
    if(QueueEmpty(q)) return  ERROR;           //队空
    p=q->front->next;
    e=p->data;
    q->front->next=p->next;
    free(p);
    if(q->front->next==NULL)  q->rear=q->front;
    return OK;
}
```

(5) 求队列长度。

```
int QueueLength(LinkQueue *q)
{
    QueueNode *p; int len=0;
    p=q->front->next;
    while(p)
    { len++;
      p=p->next;
    }
    return len;
}
```

3.4 队列应用举例

队列是一种相当实用的数据结构,可应用于生产、生活的各个方面。凡是符合先进先出运算特点,都可以使用队列来实现。

3.4.1 键盘输入循环缓冲区问题

在操作系统中,循环队列经常用于实时应用程序。例如,当程序正在执行其他任务时,用户可以从键盘上不断输入所要输入的内容。很多字处理软件就是这样工作的。系统在利用这种分时处理方法时,用户输入的内容不能在屏幕上立刻显示出来,直到当前正在工作的那个进程结束为止。但在这个进程执行时,系统是在不断地检查键盘状态,如果检测到用户输入了一个新的字符,就立刻把它存到系统缓冲区中,然后继续运行原

来的进程。当当前工作的进程结束后，系统就从缓冲区中取出输入的字符，并按要求进行处理。这里的键盘输入缓冲区采用了循环队列。队列的特性保证了输入字符先输入、先保存、先处理的要求，循环结构又有效地限制了缓冲区的大小，并避免了假溢出问题。下面用程序来模拟这种应用情况。

1. 问题的描述

假设有两个进程同时存在于一个程序中。其中第一个进程在屏幕上连续显示字符“A”，与此同时，程序不断检测键盘是否有输入，如果有，就读入用户输入的字符并保存到输入缓冲区中。在用户输入时，输入的字符并不立即回显在屏幕上。而当用户输入一个逗号(,)时，表示第一个进程结束，这时第二个进程才从缓冲区中读取那些已输入的字符并显示在屏幕上。第二个进程结束后，程序又进入第一个进程，重新显示字符“A”，同时用户又可以继续输入字符，直到用户输入一个分号(;)键，才结束第一个进程，同时也结束整个程序。

2. 具体实现

具体实现如算法 3.4 所示。

```
typedef char Elemtyp;
void keyboardinput()
{ //模拟键盘输入循环缓冲区
    char ch1, ch2;
    SqQueue Q;
    int f;
    InitQueue(Q); //队列初始化
    for(;;)
    { for(;;) //第一个进程
        { printf("A");
          if( kbhit() ) //检测键盘是否有输入
          { ch1=bdos(7,0,0); //通过 DOS 命令读入一个字符
            f=EnQueue(Q, ch1);
            if(f==ERROR)
            { printf("循环队列已满\n");
              break; //循环队列满时,强制中断第一个进程
            }
          }
        }
        if(ch1==',' || ch1==';')
            break; //第一个进程正常结束
    }
    while (!QueueEmpty(Q)) //第二个进程
    {
        DeQueue(Q, ch2);
        putchar(ch2); //显示输入缓冲区的内容
    }
}
```